```
!pip install -q datasets sentence-transformers scikit-learn pandas matplotlib

from datasets import load_dataset
import pandas as pd

ds = load_dataset("cnamuangtoun/resume-job-description-fit")
print(ds)

train = ds["train"].to_pandas()
test = ds["test"].to_pandas()

print("\nTrain shape:", train.shape)
print("Test shape:", test.shape)
print("\nColumns:", train.columns.tolist())
print("\nLabel values:", train["label"].unique())
print("\nLabel counts (train):")
print(train["label"].value_counts())
print("\nLabel counts (test):")
print(test["label"].value_counts())
```

```
DatasetDict({
    train: Dataset({
        features: ['resume_text', 'job_description_text', 'label'],
        num_rows: 6241
    })
    test: Dataset({
        features: ['resume_text', 'job_description_text', 'label'],
        num_rows: 1759
    })
})

Train shape: (6241, 3)
Test shape: (1759, 3)

Columns: ['resume_text', 'job_description_text', 'label']

Label values: ['No Fit' 'Potential Fit' 'Good Fit']

Label counts (train):
label
No Fit           3143
Potential Fit    1556
Good Fit         1542
Name: count, dtype: int64

Label counts (test):
label
No Fit           857
Good Fit         458
Potential Fit    444
Name: count, dtype: int64
```

```
label_map_binary = {"Good Fit": 1, "Potential Fit": 0, "No Fit": 0}
label_map_graded  = {"Good Fit": 2, "Potential Fit": 1, "No Fit": 0}

test["label_bin"] = test["label"].map(label_map_binary)
test["label_graded"] = test["label"].map(label_map_graded)

print(test["label_bin"].value_counts())
print(test["label_graded"].value_counts())
```

```
label_bin
0    1301
1     458
Name: count, dtype: int64
label_graded
0    857
2    458
1    444
Name: count, dtype: int64
```

```
print("Total test rows:", len(test))

# peek at a few job descriptions to see what job families exist
for jd in test["job_description_text"].sample(5, random_state=1):
    print(jd[:150])
    print("---")
```

```
Total test rows: 1759
Position Type

Full time
```

```
Type Of Hire

Experienced (relevant combo of work and education)

Education Desired

Bachelor of Computer Engineering

Trave
---
Our client is looking for experienced professional with a strong background in healthcare, including Pharmacy Benefit Manage
---
Primary Location: Melbourne, Florida
V-Soft Consulting is currently hiring for an Application Software Engineering V for our premier client in Melbour
---
Role: Java Developer Duration: 12 months Required Skills Developed web application using Java, J2EE, HTML, CSS, JavaScript,
---
Who we are?For the past 20 years, we have powered many Digital Experiences for the Fortune 500. Since 1999, we have grown fr
---
```

```
import re

def contains_keywords(text, keywords):
    text = str(text).lower()
    return any(k in text for k in keywords)

ba_keywords = ["business analyst"]

test["is_ba_job"] = test["job_description_text"].apply(lambda x: contains_keywords(x, ba_keywords))
ba_jobs = test[test["is_ba_job"]]

print("Rows with Business Analyst job postings:", len(ba_jobs))
print("Unique BA job descriptions:", ba_jobs["job_description_text"].nunique())
print(ba_jobs["label"].value_counts())

ba_group_sizes = ba_jobs.groupby("job_description_text").size().sort_values(ascending=False)
print(ba_group_sizes)
```

```
Rows with Business Analyst job postings: 240
Unique BA job descriptions: 10
label
No Fit           120
Good Fit          61
Potential Fit     59
Name: count, dtype: int64
job_description_text
ServiceNow BA perspective, here are some of the requirements: Years of experience: 8+ yrs as a Business Analyst. Scrum Maste
Position: Business Analyst IVLocation: St. Louis, MO (Onsite from day one)Duration: 6+ Month Contract to Hire Job Descriptic
About Grainger\nGrainger is a leading broad line distributor with operations primarily in North America, Japan and the Unite
Additional Job Description\nJob Title: Business Analyst (with Pega OR Rule Engine tool experience)\nLocation: St. Louis, MO\
Gaming Business Analyst\n Experience: 5+ yrs\n Location: Bellevue, USA\n\nResponsibilities:\n Ability to understand vast amo
Assignment Detail\n\nIndustry:IT ConsultingServices\n\nJob Description\n\nBusiness Analyst-Level III\n\nJob Number: 23-02173
JOB TITLE: Business Analyst\n\nDoes using data science to solve business problems sound intriguing? Enjoy working collaborat
ABOUT THE COMPANYFounded in 1888, Lee Kum Kee, a Hong Kong-based global food company, specializes in creating condiments and
Role  Business Analyst (AI ML)Location  Remote Duration - 6+ months\nJob Summary: The Jr. Business Analyst (AI  ML) will be
About the role:  DUAL North America, Inc. Is seeking a Business Analyst, Property to join the Capacity team. The Property Bu
dtype: int64
```

```
skills_dict = {
    "sql": ["sql"],
    "excel": ["excel"],
    "requirements gathering": ["requirements gathering", "requirement gathering", "business requirement"],
    "stakeholder management": ["stakeholder", "stakeholders"],
    "agile": ["agile", "scrum"],
    "documentation": ["documentation", "use cases", "user stories"],
    "process improvement": ["process improvement", "process mapping", "process optimization"],
    "communication": ["communication", "communicate"],
    "project management": ["project management", "project plan"],
    "data analysis": ["data analysis", "analytical", "analytics"],
}
required_skills = list(skills_dict.keys())

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[^a-z0-9\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def extract_skills(text, skills_dict):
    text_clean = clean_text(text)
    found = set()
    for skill, aliases in skills_dict.items():
        if any(alias in text_clean for alias in aliases):
```

```
                found.add(skill)
    return found
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer
import numpy as np

emb_model = SentenceTransformer('all-MiniLM-L6-v2')

def precision_at_k(rel, k): return sum(rel[:k]) / k
def recall_at_k(rel, k, total): return sum(rel[:k]) / total if total else 0
def dcg_at_k(rel, k): return sum(r / np.log2(i + 2) for i, r in enumerate(rel[:k]))
def ndcg_at_k(rel, k):
    ideal = sorted(rel, reverse=True)
    idcg = dcg_at_k(ideal, k)
    return dcg_at_k(rel, k) / idcg if idcg else 0

# only evaluate jobs with enough candidates to make ranking meaningful
MIN_GROUP_SIZE = 10
K = 5

job_groups = [g for g in ba_jobs.groupby("job_description_text") if len(g[1]) >= MIN_GROUP_SIZE]
print(f"Evaluating {len(job_groups)} job postings with >= {MIN_GROUP_SIZE} candidates each")

results_tfidf = []
results_embed = []
all_scored_groups = []  # keep for explainability demo later

for job_text, group in job_groups:
    group = group.reset_index(drop=True)
    resumes = group["resume_text"].apply(lambda x: str(x)[:3000]).tolist()

    # TF-IDF: fit per job group (job + its candidate resumes)
    vectorizer = TfidfVectorizer(stop_words="english", max_features=3000)
    corpus = [clean_text(job_text)] + [clean_text(r) for r in resumes]
    tfidf_matrix = vectorizer.fit_transform(corpus)
    tfidf_sims = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:]).flatten()

    # Embeddings
    job_emb = emb_model.encode([job_text[:3000]])
    resume_embs = emb_model.encode(resumes)
    embed_sims = cosine_similarity(job_emb, resume_embs).flatten()

    group["tfidf_score"] = tfidf_sims
    group["embed_score"] = embed_sims
    group["label_bin"] = group["label"].map({"Good Fit": 1, "Potential Fit": 0, "No Fit": 0})
    group["label_graded"] = group["label"].map({"Good Fit": 2, "Potential Fit": 1, "No Fit": 0})

    total_relevant = group["label_bin"].sum()
    if total_relevant == 0:
        continue  # skip jobs with no "Good Fit" at all, can't measure recall

    for score_col, results_list in [("tfidf_score", results_tfidf), ("embed_score", results_embed)]:
        ranked = group.sort_values(score_col, ascending=False)
        rel_bin = ranked["label_bin"].tolist()
        rel_graded = ranked["label_graded"].tolist()
        results_list.append({
            "job": job_text[:60],
            "n_candidates": len(group),
            f"P@{K}": precision_at_k(rel_bin, K),
            f"R@{K}": recall_at_k(rel_bin, K, total_relevant),
            f"NDCG@{K}": ndcg_at_k(rel_graded, K),
        })

    all_scored_groups.append(group)

df_tfidf_results = pd.DataFrame(results_tfidf)
df_embed_results = pd.DataFrame(results_embed)

print("\n=== TF-IDF — averaged across", len(df_tfidf_results), "jobs ===")
print(df_tfidf_results[[f"P@{K}", f"R@{K}", f"NDCG@{K}"]].mean())

print("\n=== Embedding — averaged across", len(df_embed_results), "jobs ===")
print(df_embed_results[[f"P@{K}", f"R@{K}", f"NDCG@{K}"]].mean())
```

```
Loading weights: 100%                                                    103/103 [00:00<00:00, 486.29it/s]
Evaluating 7 job postings with >= 10 candidates each

=== TF-IDF — averaged across 3 jobs ===
P@5       0.466667
R@5       0.406349
NDCG@5    0.676548
dtype: float64

=== Embedding — averaged across 3 jobs ===
P@5       0.466667
R@5       0.406349
NDCG@5    0.699751
dtype: float64
```

```
best_group = max(all_scored_groups, key=len)

# find the matching job text correctly
demo_job_text = None
for job_text, group in job_groups:
    if len(group) == len(best_group) and group["label"].tolist() == best_group["label"].tolist():
        demo_job_text = job_text
        break

required = extract_skills(demo_job_text, skills_dict)
print("DEMO JOB (first 200 chars):", demo_job_text[:200])
print("Required skills detected in JD:", required)
print()

for _, row in best_group.sort_values("tfidf_score", ascending=False).head(5).iterrows():
    resume_skills = extract_skills(row["resume_text"], skills_dict)
    matched = required & resume_skills
    missing = required - resume_skills
    print(f"Label: {row['label']} | TF-IDF: {row['tfidf_score']:.3f} | Embed: {row['embed_score']:.3f}")
    print(f"Matched: {matched} | Missing: {missing}")
    print()
```

```
DEMO JOB (first 200 chars): ServiceNow BA perspective, here are some of the requirements: Years of experience: 8+ yrs as a B
Required skills detected in JD: {'process improvement', 'communication', 'documentation', 'requirements gathering', 'agile',

Label: Good Fit | TF-IDF: 0.213 | Embed: 0.588
Matched: {'stakeholder management', 'communication', 'documentation', 'agile', 'excel', 'requirements gathering'} | Missing:

Label: No Fit | TF-IDF: 0.190 | Embed: 0.521
Matched: {'communication', 'excel', 'process improvement', 'data analysis'} | Missing: {'agile', 'stakeholder management', '

Label: No Fit | TF-IDF: 0.149 | Embed: 0.503
Matched: {'process improvement', 'stakeholder management', 'communication', 'documentation', 'agile', 'excel', 'requirements

Label: No Fit | TF-IDF: 0.148 | Embed: 0.512
Matched: {'documentation', 'agile', 'excel', 'requirements gathering', 'data analysis'} | Missing: {'communication', 'proces

Label: No Fit | TF-IDF: 0.146 | Embed: 0.458
Matched: {'process improvement', 'requirements gathering', 'data analysis'} | Missing: {'communication', 'documentation', 'a
```

```
job_description_B = """We are looking for a Business Analyst skilled in requirements gathering, stakeholder
management, and process improvement. The candidate should be experienced with Agile methodology, strong in
Excel, capable of producing clear documentation, and possess excellent communication skills. Data analysis
experience is a plus."""

concise_phrases = {
    "requirements gathering": "requirements gathering",
    "stakeholder management": "stakeholder management",
    "process improvement": "process improvement",
    "agile": "agile",
    "excel": "excel",
    "documentation": "documentation",
    "communication": "communication",
    "data analysis": "data analysis",
}

verbose_phrases = {
    "requirements gathering": "eliciting and formally capturing business needs from multiple departments",
    "stakeholder management": "building strong working relationships with cross-functional partners and leadership",
    "process improvement": "redesigning existing workflows to increase operational efficiency",
    "agile": "working within iterative sprint-based delivery frameworks",
    "excel": "advanced spreadsheet modeling and reporting tools",
    "documentation": "producing detailed functional specifications and user stories",
    "communication": "clearly conveying complex findings to both technical and non-technical audiences",
    "data analysis": "interpreting large datasets to uncover actionable business insights",
}
```

```
required_skills_B = list(concise_phrases.keys())

paired_skillsets = [
    required_skills_B,
    ["requirements gathering","stakeholder management","excel","communication"],
    ["agile","excel","documentation","communication"],
    ["requirements gathering","process improvement","data analysis"],
    ["stakeholder management","agile","documentation"],
    ["excel","communication","data analysis"],
    required_skills_B,
    ["requirements gathering","stakeholder management","process improvement","agile"],
    ["documentation","communication","data analysis"],
    ["requirements gathering","excel","agile","communication"],
    ["stakeholder management","process improvement","excel"],
    ["requirements gathering","stakeholder management","agile","excel","documentation"],
    ["communication","data analysis","process improvement"],
    ["requirements gathering","documentation","communication"],
    required_skills_B,
]

def make_concise(name, skills):
    phrases = ", ".join(concise_phrases[s] for s in skills)
    return f"{name} has hands-on professional experience with {phrases}. Delivered results across multiple projects in prev

def make_verbose(name, skills):
    phrases = "; ".join(verbose_phrases[s] for s in skills)
    return (f"{name} is a dedicated professional who, over several years in industry, developed strong "
            f"capability in {phrases}. {name} is known for a proactive, detail-oriented approach to "
            f"solving complex business problems and consistently delivering value to stakeholders.")

pairs = []
for i, skills in enumerate(paired_skillsets, start=1):
    name = f"Profile_{i}"
    pairs.append({
        "name": name,
        "skills": skills,
        "concise_text": make_concise(name, skills),
        "verbose_text": make_verbose(name, skills),
    })

df_B = pd.DataFrame(pairs)
df_B.head()
```

	name	skills	concise_text	verbose_text
0	Profile_1	[requirements gathering, stakeholder managemen...	Profile_1 has hands-on professional experience...	Profile_1 is a dedicated professional who, ove...
1	Profile_2	[requirements gathering, stakeholder managemen...	Profile_2 has hands-on professional experience...	Profile_2 is a dedicated professional who, ove...
2	Profile_3	[agile, excel, documentation, communication]	Profile_3 has hands-on professional experience...	Profile_3 is a dedicated professional who, ove...

Next steps: Generate code with df_B | New interactive sheet

```
def tfidf_scores(job_text, resume_texts):
    vectorizer = TfidfVectorizer(stop_words='english')
    corpus = [clean_text(job_text)] + [clean_text(t) for t in resume_texts]
    tfidf_matrix = vectorizer.fit_transform(corpus)
    return cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:]).flatten()

def embedding_scores(job_text, resume_texts):
    job_emb = emb_model.encode([job_text])
    resume_embs = emb_model.encode(resume_texts)
    return cosine_similarity(job_emb, resume_embs).flatten()

df_B["tfidf_concise"] = tfidf_scores(job_description_B, df_B["concise_text"].tolist())
df_B["tfidf_verbose"] = tfidf_scores(job_description_B, df_B["verbose_text"].tolist())
df_B["embed_concise"] = embedding_scores(job_description_B, df_B["concise_text"].tolist())
df_B["embed_verbose"] = embedding_scores(job_description_B, df_B["verbose_text"].tolist())

df_B["tfidf_diff"] = (df_B["tfidf_concise"] - df_B["tfidf_verbose"]).abs()
df_B["embed_diff"] = (df_B["embed_concise"] - df_B["embed_verbose"]).abs()

df_B["rank_concise_tfidf"] = df_B["tfidf_concise"].rank(ascending=False)
df_B["rank_verbose_tfidf"] = df_B["tfidf_verbose"].rank(ascending=False)
df_B["rank_changed_tfidf"] = df_B["rank_concise_tfidf"] != df_B["rank_verbose_tfidf"]

df_B["rank_concise_embed"] = df_B["embed_concise"].rank(ascending=False)
```

```
df_B["rank_verbose_embed"] = df_B["embed_verbose"].rank(ascending=False)
df_B["rank_changed_embed"] = df_B["rank_concise_embed"] != df_B["rank_verbose_embed"]

print("=== WRITING STYLE AUDIT (n =", len(df_B), ") ===")
print(f"TF-IDF mean |score change|: {df_B['tfidf_diff'].mean():.4f}")
print(f"Embedding mean |score change|: {df_B['embed_diff'].mean():.4f}")
print(f"TF-IDF % rank changed: {df_B['rank_changed_tfidf'].mean()*100:.1f}%")
print(f"Embedding % rank changed: {df_B['rank_changed_embed'].mean()*100:.1f}%")

df_B[["name","tfidf_concise","tfidf_verbose","tfidf_diff",
      "embed_concise","embed_verbose","embed_diff",
      "rank_changed_tfidf","rank_changed_embed"]]
```

```
=== WRITING STYLE AUDIT (n = 15 ) ===
TF-IDF mean |score change|: 0.1866
Embedding mean |score change|: 0.0804
TF-IDF % rank changed: 93.3%
Embedding % rank changed: 80.0%
```

	name	tfidf_concise	tfidf_verbose	tfidf_diff	embed_concise	embed_verbose	embed_diff	rank_changed_tfidf	rank_c
0	Profile_1	0.349082	0.034893	0.314190	0.404248	0.383484	0.020764	True	
1	Profile_2	0.210810	0.025700	0.185110	0.408866	0.323814	0.085051	True	
2	Profile_3	0.162020	0.028974	0.133046	0.416964	0.361855	0.055109	True	
3	Profile_4	0.233531	0.027460	0.206071	0.434228	0.337211	0.097017	True	
4	Profile_5	0.172205	0.036818	0.135387	0.379580	0.308668	0.070913	True	
5	Profile_6	0.170460	0.021084	0.149375	0.461972	0.366223	0.095749	False	
6	Profile_7	0.349082	0.034893	0.314190	0.464696	0.417737	0.046959	True	
7	Profile_8	0.250302	0.025136	0.225166	0.414056	0.321837	0.092219	True	
8	Profile_9	0.173966	0.036838	0.137128	0.390767	0.323332	0.067435	True	
9	Profile_10	0.152318	0.016537	0.135781	0.459184	0.315936	0.143248	True	
10	Profile_11	0.171186	0.017626	0.153561	0.419982	0.308408	0.111574	True	
11	Profile_12	0.205838	0.032466	0.173372	0.439207	0.330219	0.108988	True	
12	Profile_13	0.175617	0.016815	0.158801	0.409435	0.323258	0.086177	True	
13	Profile_14	0.128962	0.030937	0.098026	0.407604	0.319031	0.088572	True	
14	Profile_15	0.312038	0.031546	0.280491	0.411822	0.375605	0.036217	True	

```
skills_dict_B = {k: [v] for k, v in concise_phrases.items()}

rows = []
for _, r in df_B.iterrows():
    found_verbose = extract_skills(r["verbose_text"], skills_dict_B)
    missed = set(r["skills"]) - found_verbose
    rows.append({"name": r["name"], "true_skills": r["skills"], "missed_in_verbose": sorted(missed)})

df_skill_audit = pd.DataFrame(rows)
df_skill_audit["num_missed"] = df_skill_audit["missed_in_verbose"].apply(len)
print(f"Profiles with at least one skill undetected after rewording: "
      f"{(df_skill_audit['num_missed']>0).sum()} / {len(df_skill_audit)}")
df_skill_audit.sort_values("num_missed", ascending=False)
```

```
Profiles with at least one skill undetected after rewording: 15 / 15
```

	name	true_skills	missed_in_verbose	num_missed
0	Profile_1	[requirements gathering, stakeholder managemen...	[agile, communication, data analysis, document...	8
6	Profile_7	[requirements gathering, stakeholder managemen...	[agile, communication, data analysis, document...	8
14	Profile_15	[requirements gathering, stakeholder managemen...	[agile, communication, data analysis, document...	8
11	Profile_12	[requirements gathering, stakeholder managemen...	[agile, documentation, excel, requirements gat...	5
7	Profile_8	[requirements gathering, stakeholder managemen...	[agile, process improvement, requirements gath...	4
2	Profile_3	[agile, excel, documentation, communication]	[agile, communication, documentation, excel]	4
1	Profile_2	[requirements gathering, stakeholder managemen...	[communication, excel, requirements gathering,...	4
9	Profile_10	[requirements gathering, excel, agile, communi...	[agile, communication, excel, requirements gat...	4
5	Profile_6	[excel, communication, data analysis]	[communication, data analysis, excel]	3
3	Profile_4	[requirements gathering, process improvement, ...	[data analysis, process improvement, requireme...	3
4	Profile_5	[stakeholder management, agile, documentation]	[agile, documentation, stakeholder management]	3
10	Profile_11	[stakeholder management, process improvement, ...	[excel, process improvement, stakeholder manag...	3
8	Profile_9	[documentation, communication, data analysis]	[communication, data analysis, documentation]	3
12	Profile_13	[communication, data analysis, process improve...	[communication, data analysis, process improve...	3
13	Profile_14	[requirements gathering, documentation, commun...	[communication, documentation, requirements ga...	3

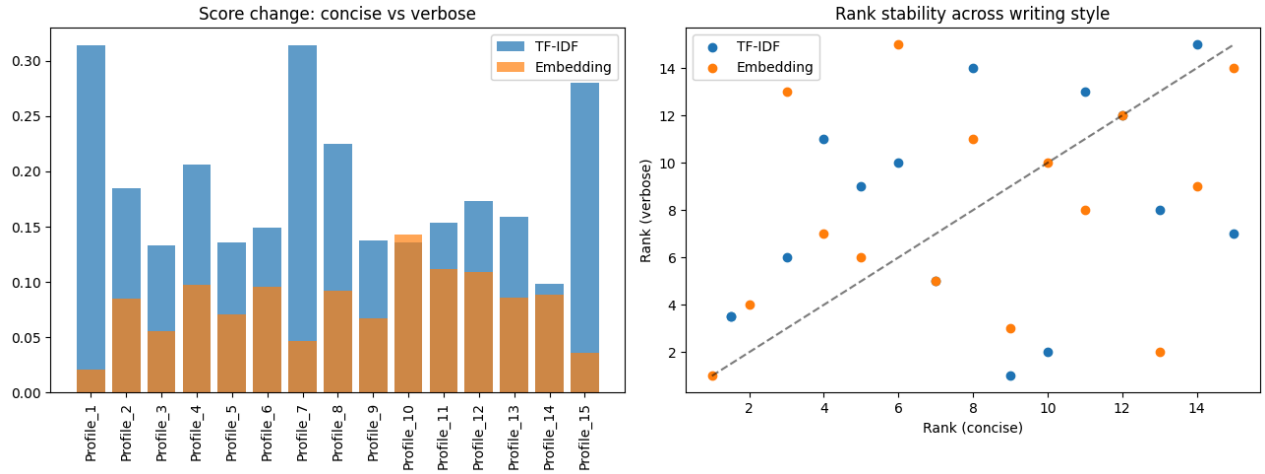
Gemini

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(13,5))
axes[0].bar(df_B["name"], df_B["tfidf_diff"], label="TF-IDF", alpha=0.7)
axes[0].bar(df_B["name"], df_B["embed_diff"], label="Embedding", alpha=0.7)
axes[0].set_xticklabels(df_B["name"], rotation=90)
axes[0].set_title("Score change: concise vs verbose")
axes[0].legend()

axes[1].scatter(df_B["rank_concise_tfidf"], df_B["rank_verbose_tfidf"], label="TF-IDF")
axes[1].scatter(df_B["rank_concise_embed"], df_B["rank_verbose_embed"], label="Embedding")
axes[1].plot([1,len(df_B)],[1,len(df_B)], 'k--', alpha=0.5)
axes[1].set_xlabel("Rank (concise)"); axes[1].set_ylabel("Rank (verbose)")
axes[1].set_title("Rank stability across writing style")
axes[1].legend()
plt.tight_layout(); plt.show()
```

```
/tmp/ipykernel_2149/1558905529.py:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. af
  axes[0].set_xticklabels(df_B["name"], rotation=90)
```




```
worst = df_B.sort_values("tfidf_diff", ascending=False).iloc[0]
print("EXAMPLE — Largest TF-IDF score drop due to rewording:")
print(f"Profile: {worst['name']}")
print(f"True skills: {worst['skills']}")
print(f"\nConcise version:\n{worst['concise_text']}")
print(f"TF-IDF score: {worst['tfidf_concise']:.3f}")
```

```
print(f"\nVerbose version:\n{worst['verbose_text']}")
print(f"TF-IDF score: {worst['tfidf_verbose']:.3f}")
```

```
EXAMPLE — Largest TF-IDF score drop due to rewording:
```